

RGPV Study Agent

Software Requirements Specification (SRS)

Version 1.0 | July 2026

Author: Ayush Pathak | Status: In Development

A zero-cost, privacy-first RAG-based academic assistant for RGPV engineering students. Delivers exam-ready structured answers mapped to university grading rubrics (4-Mark / 7-Mark) from course PDFs, powered by student-supplied API keys.

1. Introduction

1.1 Purpose

This document defines the complete software requirements for the RGPV Study Agent — a Retrieval-Augmented Generation (RAG) system that helps engineering students prepare for semester exams by generating structured, rubric-aligned answers from their own course notes.

1.2 Problem Statement

Students using standard LLMs face three critical problems:

- Generic answers that don't match RGPV word/line count requirements for 4-mark and 7-mark questions.
- Vague blending of overlapping topics across subjects with no source attribution.
- No memory of context — every question starts fresh with no awareness of follow-ups.

1.3 Solution Overview

The RGPV Study Agent solves this by:

- Ingesting subject PDFs into a vector database with semantic chunking and metadata tagging.
- Auto-detecting answer format (short / 4-mark / 7-mark) from student query intent.
- Enforcing structured output via constitutional prompt templates inspired by Anthropic's Claude prompting approach.
- Maintaining rolling memory (last Q+A) without ballooning token usage.
- Using BYOK (Bring Your Own Key) so hosting costs remain zero.

1.4 Scope

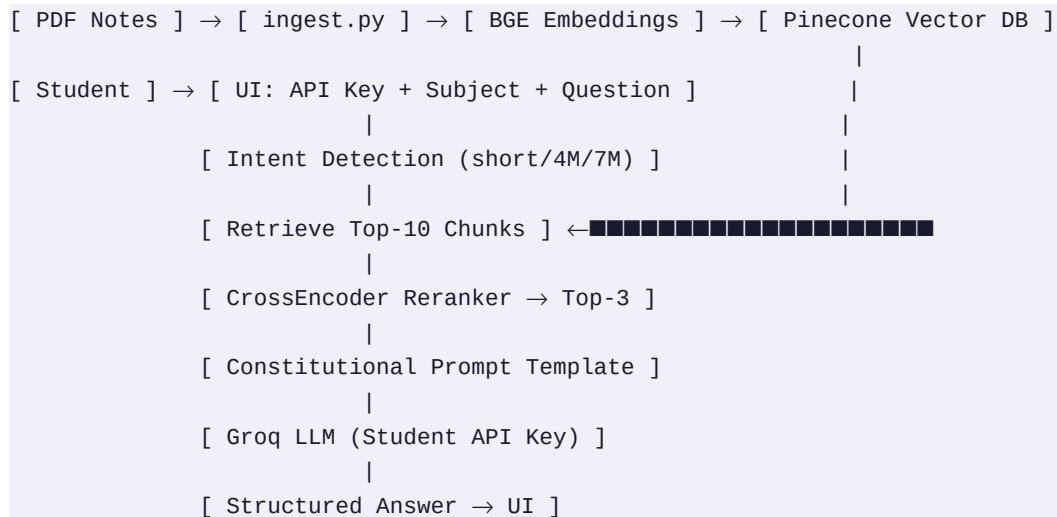
Phase 1 (current): Single-user local deployment for personal exam preparation. Phase 2: Multi-user HuggingFace Spaces deployment with BYOK for 1000+ RGPV students.

2. Overall Description

2.1 Product Perspective

The system is a standalone web application with a Python backend (Flask/Streamlit), a vector database (Pinecone serverless), and a frontend chat UI. It delegates LLM inference to the student's own API key (Groq/OpenAI/Gemini).

2.2 System Architecture



2.3 User Classes

- Primary User: RGPV engineering students preparing for mid-sem and end-sem exams.
- Admin/Developer: Ayush Pathak — manages PDF corpus, deploys updates, monitors quality.

2.4 Operating Environment

- Hosting: HuggingFace Spaces (Free Tier — 16GB RAM, 2 vCPU)
- Vector DB: Pinecone Serverless Free Tier (100K vectors)
- LLM Inference: Groq API via student's own key (Llama 3.3 70B)
- Embeddings: BAAI/bge-small-en-v1.5 (local, runs on CPU)
- Reranker: cross-encoder/ms-marco-MiniLM-L-6-v2 (local, runs on CPU)

3. Technical Stack

Layer	Tool	Reason
PDF Ingestion	LlamaIndex + pypdf	Reliable PDF parsing, node pipeline
Embeddings	BAAI/bge-small-en-v1.5	Best retrieval quality on CPU
Vector Store	Pinecone Serverless	Free, persistent, no infra cost
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Re-scores top-10 → top-3 accurately
LLM	Groq — Llama 3.3 70B	Fast inference, free tier, BYOK
Backend	Flask → Streamlit	Streamlit for simpler BYOK UI
Memory	Rolling summary (60 words)	Minimal token overhead per request
Chat History	SQLite (local)	Free, private, no cloud DB needed
Hosting	HuggingFace Spaces	16GB RAM free, Python-native

4. Functional Requirements

4.1 PDF Ingestion Pipeline

- Load PDFs from data/pdfs directory using SimpleDirectoryReader.
- Chunk using SentenceSplitter: chunk_size=512, chunk_overlap=128.
- Inject metadata per chunk: filename, subject, unit, page_no.
- Embed using BGE-small, store in Pinecone with metadata.
- Incremental ingestion: detect new files, only embed new chunks.

4.2 Intent Detection

- Auto-detect answer format from question using keyword matching + fuzzy matching (difflib).
- Short (2-mark): keywords — short, brief, define, what is, summarize, 2 mark.
- 4-mark: keywords — 4 mark, midsem, medium, longer, moderate.
- 7-mark: keywords — 7 mark, detail, endsem, explain in detail, long answer.
- Typo tolerance via get_close_matches() with 0.8 cutoff.
- Default: 7-mark if no intent detected.

4.3 Retrieval Pipeline

- Embed student query using BGE-small.
- Retrieve top-10 chunks from Pinecone by cosine similarity.
- Re-score all 10 chunks using CrossEncoder reranker.
- Filter: discard chunks with reranker score below 0.5.
- If no chunks pass threshold: return fallback message using general LLM knowledge.
- Pass top-3 ranked chunks as context to LLM.

4.4 Memory System

- Store last Q+A exchange as rolling summary (max 60 words of answer).
- Prepend memory to retrieval query for follow-up questions.
- Detect pure intent queries (3 words or less) — extract topic from memory for retrieval.
- Detect meta-queries (what did I ask, previous question) — answer directly from memory without hitting retriever.
- Memory stored per Flask/Streamlit session, resets on browser close.

4.5 Prompt Templates

- Three constitutional prompt templates: SHORT, FOUR_MARK, SEVEN_MARK.
- Each prompt uses XML tags: identity, rules, task, output format.
- Rules enforce: no repetition, no hallucination, stop after conclusion, output only the answer.
- Context passed inside context XML tags to prevent leaking into output.

4.6 Answer Formats

Short (2-mark): Definition → Key Points (3) → Example → Conclusion. Max 8 lines.

4-mark: Definition + Simply → Key Points (4) → Example → Conclusion. ~1 page.

7-mark: Definition → Why Needed → Working (5 steps) → Architecture → Types → Advantages (5) → Applications (5) → Mnemonic → Conclusion. 2-3 pages.

4.7 BYOK (Bring Your Own Key)

- Student enters Groq API key in UI sidebar.
- Key passed to ChatGroq per session, never stored server-side.
- Error handling: invalid key, expired key, quota exceeded — friendly UI messages.

4.8 Chat History

- Store full session conversations in local SQLite database on student's device.
- No cloud storage — privacy-first, zero backend DB cost.
- Student can view, search, and clear history from UI.

5. Non-Functional Requirements

5.1 Performance

- Embedding a query: under 1 second on CPU (BGE-small).
- Retrieval + reranking: under 3 seconds for top-10 → top-3.
- LLM generation (Groq): under 5 seconds for 7-mark answer.
- Total response time target: under 10 seconds end-to-end.

5.2 Scalability

- Pinecone free tier supports up to 100K vectors (~5000 PDF pages).
- HuggingFace Spaces handles concurrent users on 16GB RAM.
- BYOK model ensures LLM cost scales with users, not with the developer.

5.3 Privacy & Security

- No student data stored server-side.
- API keys never logged or persisted beyond the session.
- Chat history stored locally on student device (SQLite).

5.4 Reliability

- Pinecone is cloud-hosted — no data loss on HF Space restart (unlike ChromaDB).
- Fallback to general knowledge if retrieval confidence is low.
- Graceful error messages for all API failures.

6. Current Development Status

6.1 Completed

- PDF ingestion with pypdf + LlamaIndex
- BGE-small-en-v1.5 embeddings
- ChromaDB vector store (migrating to Pinecone)
- CrossEncoder reranker (top-10 → top-3)
- Intent detection with fuzzy matching
- Constitutional prompt templates (short / 4-mark / 7-mark)
- Rolling memory with topic-aware retrieval
- Meta-query handler (what did I ask)
- Incremental ingestion (new PDFs only)
- Metadata per chunk (filename, page_no, subject)

6.2 In Progress

- Pinecone migration
- LLM-assisted metadata tagging (subject, unit)
- Score threshold fallback to general knowledge
- Streamlit UI with exam mode dropdown
- BYOK — student API key input
- SQLite local chat history
- HIGH IMPORTANCE query aggregation
- Error handling for invalid/expired API keys

6.3 Known Issues

- Intent-only follow-ups (in 4 marks) still fragile without topic
- Raw chunk text occasionally leaks before formatted answer
- No RAGAS eval pipeline — quality measured manually
- PDF quality varies — some chunks contain noisy text
- No regression test suite

6.4 Future Vision

- Maths and algorithm explanations
- Flowchart generation in answers
- Multi-university support beyond RGPV
- Full RAGAS evaluation pipeline

7. Constraints & Assumptions

7.1 Constraints

- Must run entirely on free tiers — no paid infrastructure.
- LLM inference cost must be borne by the student via BYOK.
- Embedding and reranking must run on CPU (no GPU on HF free tier).
- Pinecone free tier: 100K vector limit.

7.2 Assumptions

- Students have basic familiarity with API keys.
- PDFs are in English and follow standard RGPV curriculum structure.
- Students ask one question at a time — not multi-part compound queries.
- PDF filenames contain subject/unit information for metadata extraction.

7.3 Out of Scope (v1)

- Image or diagram extraction from PDFs.
- Real-time collaborative study sessions.
- Mobile native app (web only).
- Automated PYQ (previous year question) ingestion.

8. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — combining vector search with LLM generation.
BGE	BAAI General Embeddings — embedding model optimized for retrieval tasks.
BYOK	Bring Your Own Key — students supply their own LLM API key.
CrossEncoder	A reranker model that scores query-chunk pairs for relevance.
Chunk	A segment of PDF text stored as a single vector in the database.
Intent	Detected answer format: short (2-mark), 4-mark, or 7-mark.
RAGAS	RAG Assessment framework — measures faithfulness, relevancy, context recall.
Constitutional Prompt	Structured prompt with identity, rules, task, and output format blocks.
Rolling Memory	Compressed summary of last Q+A exchange (~60 words) passed to LLM.
Pinecone	Cloud-hosted serverless vector database with free tier.
HF Spaces	HuggingFace Spaces — free Python app hosting with 16GB RAM.